Programação Avançada

Unidade 1: POO (Programação Orientada a Objetos)

Aula 01: Listas e Dicionários: Revisão e Complexidade das Operações

Prof. Eduardo Corrêa Gonçalves

04/02/2026

Sumário

Introdução

Complexidade de algoritmos (Notação “Big O”)

Listas

Operações e suas complexidades

Conceito de Método

List Comprehension

Organização em memória: array de referências

Por que o append() é O(1) ?

Dicionários

Operações e suas complexidades

Conceito de Hash

Organização em memória: HashMap

Introdução

- **Objetivos da aula**
  - Introduzir o conceito de **complexidade de algoritmos**.
  - Fazer uma breve **revisão** sobre as **operações de listas e dicionários** do Python.
  - Conhecer o **custo** (complexidade) dessas operações.
  - Conhecer alguns detalhes sobre a **forma como Python implementa** listas e dicionários.
    - Lista → Array de Referências.
    - Dicionário → HashMap.

Complexidade – A Notação "Big O" (1/3)

- **O que é "Big O" ?**
  - A notação "Big O"* é uma métrica utilizada para determinar a complexidade de um algoritmo.
  - Assumindo que *n* é o tamanho da entrada do algoritmo, a notação representa a relação entre *n* e o total de operações (ou passos) que o algoritmo irá executar para determinar a solução.
    - Por exemplo, O(n) significa que o algoritmo executa um número de operações que varia de forma linear com o tamanho de *n*.
  - **Informalmente**: Você pode entender O como "uma função da ordem de".
  - **Formalmente**: objetivo da notação "Big O" é expressar complexidade do algoritmo em termos do crescimento de número de operações que ele realiza à medida que a entrada se torna arbitrariamente grande (*n* tende a infinito).

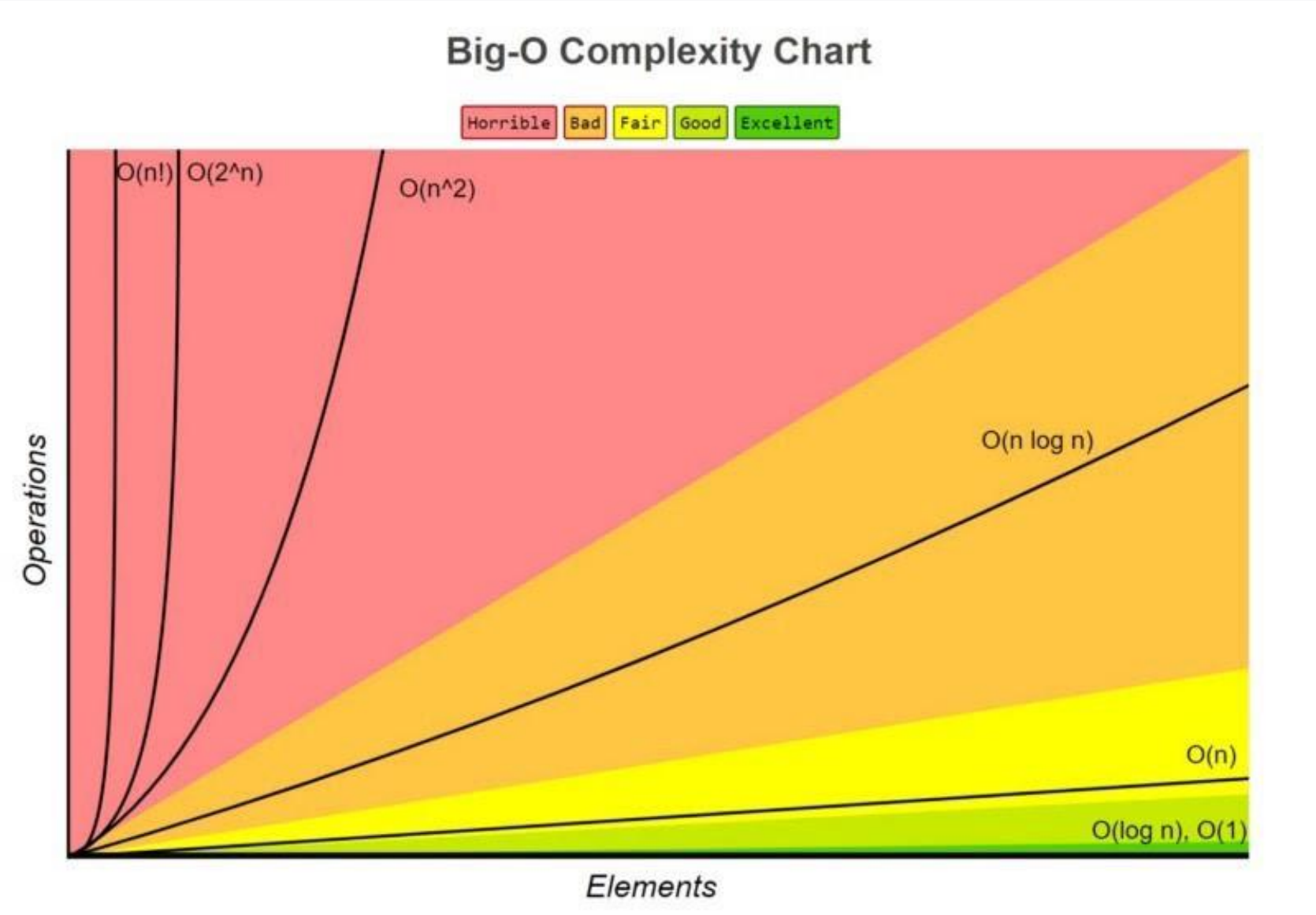
*Na verdade, o "grande O" refere-se à letra grega Omicron, tipicamente utilizada para denotar limites superiores.

Complexidade – A Notação "Big O" (2/3)

- **Complexidades computacionais mais comuns (da melhor para a pior)**

Notação	Nome	Definição	Exemplo
O(1)	Constante	Executa o mesmo nº de operações, seja qual for *n*	*Acessar elemento de uma lista pelo índice ou de um dicionário pela chave*
O(log n)	Logarítmica	nº de operações proporcional ao logaritmo de *n* (menor que *n*)	*Busca binária*
O(n)	Linear	nº de operações cresce de forma linear com *n*	*Buscar elemento pelo valor em lista não ordenada*
O(n log n)	Log Linear	Melhor que $O(n^2)$, porém não tão bom quanto O(n)	*Algoritmo de ordenação Quicksort*
$O(n^2)$	Quadrática	nº de operações proporcional ao quadrado de *n*	*Iterar sobre uma lista 2d de dimensão n x n*
$O(2^n)$	Exponencial	nº de operações dobra conforme *n* cresce.	*Fibonacci recursivo*
O(n!)	Fatorial	O pior dos piores cenários! nº de operações proporcional a *n!*	*Caixeiro viajante "ingênuo" (computar todas as rotas entre n cidades).*

Complexidade – A Notação “Big O” (3/3)



Fonte: https://medium.com/creditas-tech/algorithms-and-how-to-analyze-their-complexity-9d94d3deb380

Lista

- **Lista**
  - Coleção ordenada de *n* elementos.

[0]	[1]	[2]	[3]	
John	Yoko	Julian	Sean	*lst_familia*

  - Cada elemento está associado a um **índice**
    - Número que indica a posição do elemento na sequência.
    - O primeiro índice é *0* e o último *n-1*.
  - Características:
    - Permite elementos **duplicados**.
    - **Mutável** – pode ser alterada.
    - **Iterável** – capaz de retornar seus elementos um por vez em um laço.
    - **Sequência** – elementos possuem ordem determinada.
    - **Popular** – ED nativa mais usada do Python.
    - **Similar ao array** – mais flexível, porém menos eficiente e “chata” para operações matemáticas (não é “rápida” e nem “prática”).

Lista: Operações (1/10)

- **Operações básicas**:
  - Recuperar elemento do índice *i*
  - Modificar elemento do índice *i*
  - Verificar se elemento *x* pertence à lista
  - Iterar (percorrer todos os elementos *em ordem*)
  - Fatiar (gerar uma sublista a partir de uma lista original)
  - Inserir um elemento *x* no final (*append*)
  - Inserir um elemento *x* em outra posição
  - Remover um elemento do final (*pop*)
  - Remover um elemento de outra posição
  - Contar número de ocorrências do elemento *x*
  - Obter índice do elemento *x*
  - Inverter
  - Ordenar
  - ...

Lista: Operações (2/10)

- **Recuperando elementos pelo índice**

```
lst = ["John", "Yoko", "Julian", "Sean"]

primeiro = lst[0]              # 'John'
ultimo = lst[3]                # 'Sean'
ultimo_tambem = lst[-1]        # também retorna 'Sean'
penultimo = lst[-2]            # 'Julian'
```

- **Modificando um elemento pelo índice**

```
lst[1] = "Yoko Ono"        # ["John", "Yoko Ono", "Julian", "Sean"]
lst[0] = "John Lennon"     # ["John Lennon", "Yoko Ono", "Julian", "Sean"]
```

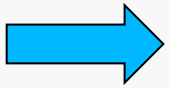
- **Verificando se elemento pertence à lista**

```
"Julian" in lst        # True
"Ringo" in lst         # False
```

Lista: Operações (3/10)

- **Iterando**

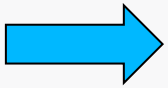
```
lst = ["John", "Yoko", "Julian", "Sean"]

for pessoa in lst:
    print(pessoa)
```



```
John
Yoko
Julian
Sean
```

- **Iterando com base nos índices**

```
for k in range(len(lst)):
    print("elemento {} = {} ".format(k, lst[k]))
```



```
elemento 0 = John
elemento 1 = Yoko
elemento 2 = Julian
elemento 3 = Sean
```

Lista: Operações (4/10)

- **Fatiando – funciona da mesma forma que a função range()**

```
lst = ["John", "Yoko", "Julian", "Sean"]

lst[0:3]        # ['John', 'Yoko', 'Julian']
lst[2:4]        # ['Julian', 'Sean']
lst[:2]         # ['John', 'Yoko']
lst[2:]         # ['Julian', 'Sean']

lst[::2]        # ['John', 'Julian']
lst[1::2]       # ['Yoko', 'Sean']

lst[-3:]        # ['Yoko', ', 'Julian', 'Sean']
lst[:-3]        # ['John']
lst[::-1]       # ['Sean', 'Julian', 'Yoko', 'John']   => retorna a lista invertida!
```

- ***Obs.**: o fatiamento **retorna uma nova lista** e não uma visão da lista original.*

Lista: Operações (5/10)

- **Repetição e Concatenação – operadores * e +**

```
a = ["John", "Paul"]
b = ["George", "Ringo"]

a * 3          # ['John', 'Paul', 'John', 'Paul', 'John', 'Paul']
a + b          # ['John', 'Paul', 'George', 'Ringo']
```

- **Obtendo o tamanho da lista – função len()**

```
a = ["John", "Paul"]
len(a)         # 2
```

- **Comparação de Listas – é feita de forma <u>lexicográfica</u>**

```
[1,2,3] > [1, 5, 10]          # False
[20] > [10, 998, 800]         # True
```

Lista: Operações (6/10)

- **Métodos disponíveis para lista**
  - lista.**append(*x*)**: insere elemento *x* no final da lista.
  - lista.**insert(*i, x*)**: insere elemento *x* na posição *i*.
  - lista.**pop*(i)*:** remove um elemento no final (caso *i* não seja especificado) ou da posição *i* da lista (caso *i* seja especificado). O método retorna o elemento removido.
  - lista.**remove(*x*)**: remove o primeiro elemento que tiver o valor *x*.
  - lista.**clear()**: esvazia a lista
  - lista.**extend(*lista2*)**: concatena os elementos da *lista2* (ou outro iterável) ao final de *lista*. Também pode-se usar *lista* += *lista2*.
  - lista.**count(*x*)*:*** conta o número de ocorrências do elemento *x.*
  - lista.**index(*x, inicio, fim*)**: obtém o índice da primeira ocorrência do elemento *x* na lista inteira ou dentro da faixa especificada em *início* e *fim*.
  - lista.**reverse()**: inverte a ordem da lista.
  - lista.**sort(*reverse=False/True*)**: ordena a lista em ordem ascendente de acordo com os valores dos elementos (*reverse=False*) ou descendente (*reverse=True*). O default é ascendente.
  - lista.**copy()**: faz uma *deep copy* da lista (gera um *clone* de fato e não uma *visão*).

Lista: Operações (7/10)

- **O que é método ?**
  - Quando você cria uma lista ou dicionário em um programa, você “ganha de presente” uma série de métodos que podem ser utilizados para executar operações sobre a lista.
  - Na realidade, toda lista ou dicionário é uma coleção que contém dados e métodos.
    - Dados → valores dos elementos armazenados.
    - Métodos → funções presentes “dentro” da lista e que sempre estão disponíveis para serem utilizadas pelo programador.
  - Como você verá na aula que vem, **método é um conceito de POO** (programação orientada a objetos)...
  - Listas e Dicionários possuem métodos porque **são implementados como classes** na linguagem Python.
    - *Mais detalhes na próxima aula !!!!*

Lista: Operações (8/10)

- **Chamando (executando, invocando) um método**
  - Para chamar um método, devemos adicionar um ponto (“.”) e o nome do método ao final do nome do objeto (lista ou dicionário no caso da aula de hoje).
  - Isto é chamado de “invocar” o método... invocar significa simplesmente “executar“
  - A sintaxe para invocar um método é resumida abaixo.

```
nome_objeto.método(argumentos)
```

Lista: Operações (9/10)

- **Inserindo e Removendo Elementos**
  - ***IMPORTANTE**: os **métodos modificam a lista** sem que precisemos realizar uma atribuição com o operador "="*

```
numeros = [5, 10]

numeros.append(20)        # insere 20 no final: [5, 10, 20]
numeros.append(10)        # insere 10 no final: [5, 10, 20, 10]
numeros.insert(2,15)      # insere 15 na posição 2: [5, 10, 15, 20, 10]
numeros.insert(0,10)      # insere 10 na posição 0: [10, 5, 10, 15, 20, 10]

numeros.pop()             # remove o último elemento: [10, 5, 10, 15, 20]
numeros.pop(3)            # remove o quarto elemento: [10, 5, 10, 20]
numeros.remove(10)        # remove o primeiro 10: [5, 10, 20]

numeros.extend([40, 50])  # estende a lista: [5, 10, 20, 40, 50]

del numeros[1:3]          # com del posso apagar 1 ou mais elementos: [5, 40, 50]
numeros.clear()           # esvazia a lista: [ ]
```

Lista: Operações (10/10)

- **Contando o números de ocorrências de um valor**

```
notas = [95, 70, 75, 100, 70, 65, 70, 100]
notas.count(70)          # 3  (= 3 ocorrências)
```

- **Obtendo o primeiro índice de um valor**

```
notas.index(70)          # 1
notas.index(70, 2, 8)    # 4
notas.index(65)          # 5
notas.index(50)          # ocasiona erro... 50 não está na lista
```

- **Invertendo e ordenando a Lista (a lista é <u>modificada</u>)**

```
notas.reverse()          # [100, 70, 65, 70, 100, 75, 70, 95]
notas.sort()             # [65, 70, 70, 70, 75, 95, 100, 100]
notas.sort(reverse=True) # [100, 100, 95, 75, 70, 70, 70, 65]
```

Lista Bidimensional (1/2)

- **Listas 2d – é uma lista de listas !**
  - Devemos especificar uma relação de listas separadas por vírgula entre colchetes.
    - Para “fingir” que é uma matriz, basta tratar cada lista como se fosse uma linha da matriz.

- 2 linhas e 2 colunas

```
matriz_pesos = [
    [0.58, 0.19],
    [0.33, 0.65]
]
```

- 4 linhas e 6 colunas

```
matriz_binaria = [
    [0, 1, 0, 0, 1, 0],
    [1, 0, 0, 1, 1, 1],
    [0, 0, 0, 0, 0, 1],
    [0, 1, 1, 0, 1, 0]
]
```

- Lista 2d. com 5 linhas, onde cada uma delas possui um número diferente de colunas (*mais uma prova da flexibilidade das listas*).

```
nao_retangular = [
    [8, 0, 5, 3, 0, 9],
    [2, 6, 0],
    [ ],
    [3, 8, 3],
    [4, 7]
]
```

Lista Bidimensional (2/2)

- **Recuperando elementos pelos índices**

```
>>> m = [
   [1, 2, 3],
   [4, 5, 6],
   [7, 8, 9]
]

# o 1º elemento de m
# é uma lista de números…
>>> m[0]
[1, 2, 3]

#o primeiro elemento dessa lista é 1,
#o segundo 2 e o terceiro 3
>>> m[0][0]
1
>>> m[0][1]
2
>>> m[0][2]
3
```

```
# o 2º elemento de m
# também é uma lista de números…
>>> m[1]
[4, 5, 6]

#o primeiro elemento dessa lista é 4,
#o segundo 5 e o terceiro 6
>>> m[1][0]
4
>>> m[1][1]
5
>>> m[1][2]
6
```

- **Obs.**: infelizmente **não** há uma **forma direta** de processar (recuperar ou alterar) uma **coluna inteira**.
- Isso porque, os elementos de uma coluna estão armazenados em listas diferentes.
- Porém, há uma solução com a função **zip(),** (*veremos futuramente*).

List Comprehension (1/12)

- **Conceito (1/2)**
  - Conforme sabemos, a ED lista não é muito prática para uso em operações matemáticas. Veja:

```
>>> a = [1, 2, 3]
>>> b = a * 2          #não multiplica cada elemento por 2, pois o asterisco é
>>> b                  #na verdade o operador de repetição
[1, 2, 3, 1, 2, 3]

#para resolver do jeito tradicional eu tenho que:
(i) instanciar "b;" (ii) fazer um for em "a"; (iii) fazer o cálculo e dar o append em "b".
>>> b = []
>>> for elemento in a:
        b.append(elemento * 2)
>>> b
[2, 4, 6]

#enfim, não dá para somar/subtrair/multiplicar/dividir todos os elementos de uma vez...
>>> c = a / 2
Traceback (most recent call last):
  File "<pyshell#5>", line 1, in <module>
    c = a/2
TypeError: unsupported operand type(s) for /: 'list' and 'int'
```

List Comprehension (2/12)

- **Conceito (2/2)**
  - Em algumas situações, torna-se possível contornar essa “inabilidade matemática” das listas utilizando o recurso **list comprehension**.
    - Trata-se de uma **notação matemática** que facilita a criação e o processamento de listas.
  - **Exemplo**: Suponha que você queira criar uma lista com as potências de 2 variando de 0 a 16. Isto é: [2^0, 2^1, ..., 2^{16}].
  - Forma **tradicional**:
    - lst = [1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384, 32768, 65536]
  - Usando **list comprehension**:
    - lst = [2**x **for** x **in range**(17)]
    - Literalmente: *para x variando de 0 a 16, faça cada elemento da lista igual a 2^x*
    - Elegante, compacto e bem mais parecido com a notação [2^0, 2^1, ..., 2^{16}]

List Comprehension (3/12)

- **Sintaxe Básica**
  - List comprehension **cria uma nova lista** a partir de uma definição concisa. Abaixo a sintaxe básica do comando:
    - lst = [*operação sobre x* **for** *x* **in** *coleção-fonte*]
    - Onde:
      - ***coleção-fonte***: é uma coleção de valores utilizados como fonte de dados para a geração da lista. Pode ser um **range()** ou uma **ED iterável** (lista, tupla, conjunto, etc.).
      - ***x***: recebe cada elemento da coleção-fonte.
      - ***operação sobre x***: representa o cálculo que faremos sobre x para gerar os elementos da nova lista. No exemplo anterior, fizemos 2**x.

```
lst = [2**x for x in range(17)]
```

List Comprehension (4/12)

- **Criando Listas**
  - Mais alguns exemplos de criação de listas:

Notação Matemática	List Comprehension
A = {x^3 | 0 ≤ x ≤ 10}	A = [x**3 **for** x **in range**(11)]
B = {1/2, 1/4, …, 1/10}	B = [1/x **for** x **in range**(2,11,2)]
C = {0, 0 0, 0 0, 0 0, 0 0, 0}	C = [([0]*2) **for** linha **in range**(5)]

List Comprehension (5/12)

- **Operações**
  - Veremos agora como utilizar o recurso list comprehension para facilitar a execução de dois tipos de operações aritméticas sobre listas:

    1. Operações com escalares

    2. Operações envolvendo duas listas de tamanho compatível.

List Comprehension (6/12)

- **Operações com escalares**

```
a = [1, 2, 3, 4]

b= [x+10 for x in a]        # [11, 12, 13, 14]        (somou 10 a cada elemento)
c= [x-1 for x in a]         # [0, 1, 2, 3]            (subtraiu 1 de cada elemento)
d  = [x * 2 for x in a]     # [2, 4, 6, 8]            (multiplicou todos por 2)
e  = [x / 2 for x in a]     # [0.5, 1.0, 1.5, 2.0]    (dividiu todos por 2)
```

#Você pode usar **qualquer função** ...seja de um módulo ou alguma que você criou.

#**Ex.**: módulo **math**, função **sqrt** (raiz quadrada).

```
import math
raiz  = [math.sqrt(x) for x in a]    # [1.0, 1.4142135623730951, 1.7320508075688772, 2.0]
```

List Comprehension (7/12)

- **Operações com duas listas**

```
>>> a=[1,2,3,4]
>>> b=[100,200,300,400]

# soma os elementos que estão na mesma posição nas duas listas
>>> soma = [a[i] + b[i] for i in range(len(a))]
>>> soma
[101, 202, 303, 404]
```

List Comprehension (8/12)

- **Incluindo condicionais (testes lógicos)**
  - É possível adicionar testes lógicos para **filtrar** e **transformar** dados.
    - EXEMPLO 1 - **FILTRAR**
    - Significa descartar alguns elementos.
    - Nesse caso, o **teste lógico** deverá ser incluído **no final**.

```
# gerar lista apenas com os números ímpares entre 1 e 20
>>>D = [x for x in range(20) if x%2==1]
>>> D
[1, 3, 5, 7, 9, 11, 13, 15, 17, 19]

# gera "b" a partir de "a", incluindo só os números entre 20 e 35
>>> a=[10,20,30,40]
>>> b = [x for x in a if x >= 20 and x <= 35]
>>> b
[20, 30]
```

List Comprehension (9/12)

- **Incluindo condicionais (testes lógicos)**
  - EXEMPLO 2 - **TRANSFORMAR**
    - Nesse caso, o **teste** deverá ser incluído **no início**.

```
# gera “b” a partir de “a” substituindo os números negativos por 0
>>> a = [-1, 3, 8, -5, 5]
>>> b = [x if x > 0 else 0 for x in a]
>>> b
[0, 3, 8, 0, 5]
```

List Comprehension (10/12)

- **Listas aninhadas (*nested lists*)**
  - Uma list comprehension pode ser aninhada dentro de outra. Isto é especialmente útil para manipular listas 2d. Neste exemplo, transformamos a matriz em um vetor.

```
>>> m = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
#pega cada linha (sublista) com "for linha in m"
#e depois cada elemento x da linha com "for x in linha".
>>> [item for linha in m for item in linha]
[1, 2, 3, 4, 5, 6, 7, 8, 9]

#dá até para obter a soma (ou máximo, mínimo, ...) dos elementos da matriz
>>> sum([item for linha in m for item in linha])
45
```

List Comprehension (11/12)

- **Listas aninhadas (*nested lists*)**
  - Abaixo uma comparação entre as resoluções sem e com list comprehension.

```
m = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
```

```
#sem list comprehension
v=[]
for linha in m:
    for item in linha:
        v.append(item)

print(v)
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
#com list comprehension
v = [item for linha in m for item in linha]

print(v)
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

List Comprehension (12/12)

- **Listas aninhadas (*nested lists*)**
  - Nesse exemplo, uma lista 2d é gerada a partir da combinação de elementos de duas listas, caso os mesmos não sejam iguais.

```
>>> [[x, y] for x in [1,2,3] for y in [3,1,4] if x != y]
[[1, 3], [1, 4], [2, 3], [2, 1], [2, 4], [3, 1], [3, 4]]
```

  - Veja outros exemplos em:

https://docs.python.org/3/tutorial/datastructures.html#list-comprehensions

Listas: Complexidade das Operações

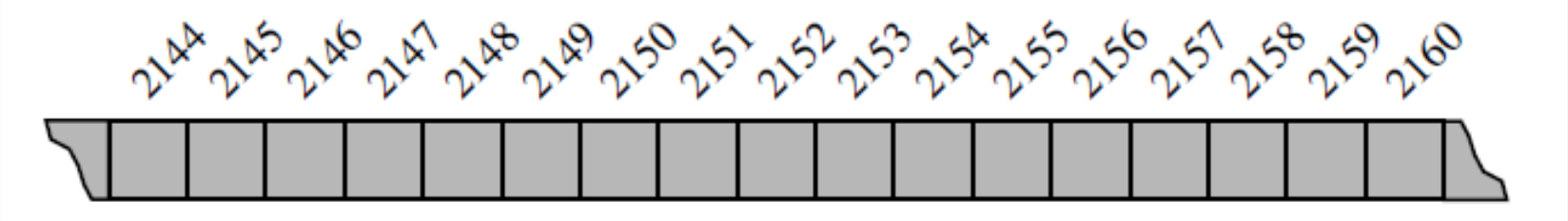
- **Complexidade**

Fonte: https://www.ics.uci.edu/~pattis/ICS-33/lectures/complexitypython.txt

```
Lists:
                                Complexity
Operation      | Example       | Class          | Notes
---------------+---------------+----------------+-------------------------------
Index          | l[i]          | O(1)           |
Store          | l[i] = 0      | O(1)           |
Length         | len(l)        | O(1)           |
Append         | l.append(5)   | O(1)           | mostly: ICS-46 covers details
Pop            | l.pop()       | O(1)           | same as l.pop(-1), popping at end
Clear          | l.clear()     | O(1)           | similar to l = []

Slice          | l[a:b]        | O(b-a)         | l[1:5]:O(l)/l[:]:O(len(l)-0)=O(N)
Extend         | l.extend(...)| O(len(...))    | depends only on len of extension
Construction   | list(...)     | O(len(...))    | depends on length of ... iterable

check ==, !=   | l1 == l2      | O(N)           |
Insert         | l[a:b] = ...  | O(N)           |
Delete         | del l[i]      | O(N)           | depends on i; O(N) in worst case
Containment    | x in/not in l| O(N)           | linearly searches list
Copy           | l.copy()      | O(N)           | Same as l[:] which is O(N)
Remove         | l.remove(...)| O(N)           |
Pop            | l.pop(i)      | O(N)           | O(N-i): l.pop(0):O(N) (see above)
Extreme value  | min(l)/max(l)| O(N)           | linearly searches list for value
Reverse        | l.reverse()   | O(N)           |
Iteration      | for v in l:   | O(N)           | Worst: no return/break in loop

Sort           | l.sort()      | O(N Log N)     | key/reverse mostly doesn't change
Multiply       | k*l           | O(k N)         | 5*l is O(N): len(l)*l is O(N**2)

Tuples support all operations that do not mutate the data structure (and they
have the same complexity classes).
```

Organização Interna de uma Lista (1/11)

- **Memória do Computador**
  - Para ser executado, um programa e os dados por ele utilizados precisam estar na **memória principal** (MP).
    - A MP é composta por um enorme número de posições (células) consecutivas.
    - Cada posição possui um **endereço** e armazena 1 byte.



  - Ou seja: cada byte está associado com um único número que serve como seu endereço.
    - Assim o sistema pode se referir e ler/alterar dados no “end. #2150” ou no “end. #2157”, etc.
  - Nos PCs modernos os endereços são representados por **64 bits**.

Organização Interna de uma Lista (2/11)

- **id()**
  - Com o uso da função **id()** é possível obter a **identidade** de um objeto Python.
    - É um número inteiro único e constante que perdurará durante o tempo de vida do objeto para uma dada execução do programa.
    - Você pode considerar que a identidade corresponde ao endereço do objeto em memória (embora não seja exatamente isso)

```
>>> lst1 = ['John', 'Paul']
>>> lst2 = ['George', 'Ringo']
>>>
>>> id(lst1)          # no seu computador vai dar outro valor
2327190461376
>>>
>>> id(lst2)          # no seu computador vai dar outro valor
2327191884352
>>>
```

    - Nos próximos slides veremos alguns detalhes sobre manipulação de listas e a organização interna das mesmas, com base nos conceitos apresentados.

Organização Interna de uma Lista (3/11)

- **Cópia Rasa x Cópia Profunda (1/2)**
  - Se você tem uma lista *a* e quiser gerar um clone *b*, **não** faça b = a.
    - A atribuição criará apenas uma nova variável *b* que irá referenciar (ou apontar para) o endereço de *a*.
    - Ou seja: *a* e *b* terão o **mesmo id** e serão a mesma lista. Basicamente *b* é apenas uma **visão** de *a*.
    - Isso é chamado de **cópia rasa** (*shallow copy*).

```
>>> a = [1,2]
>>> b = a
>>>
>>> id(a)
2327191884480
>>>
>>> id(b)
2327191884480
>>>
```

Organização Interna de uma Lista (4/11)

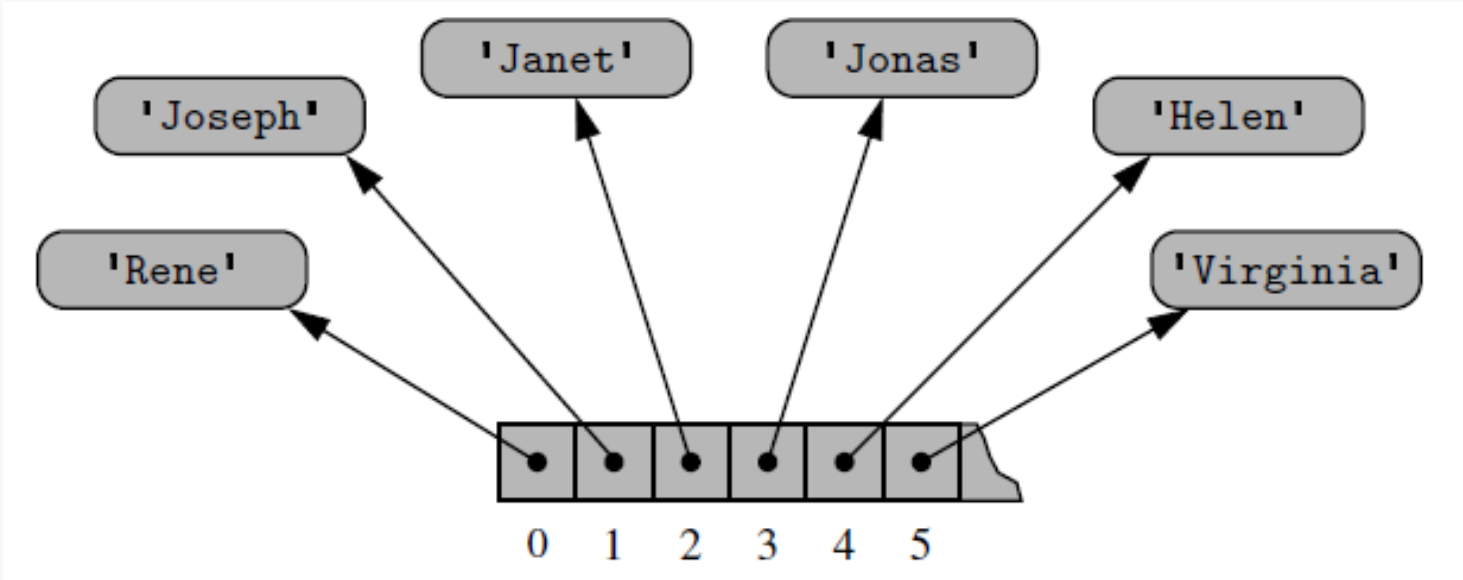
- **Cópia Rasa x Cópia Profunda (2/2)**
  - Para clonar de verdade, use um dos métodos abaixo:
    - b = a[:] *# o resultado do fatiamento é sempre um clone*
    - b = a.**copy()** *# o método copy gera um clone*
    - Neste caso, *a* e *b* serão listas diferentes, portanto, com ids diferentes.
    - Isso é chamado de **cópia profunda** (*deep copy*).

```
>>> a = [1,2]
>>> b = a[:]
>>>
>>> id(a)
2327191885184
>>>
>>> id(b)
2327191884672
>>>
```

Organização Interna de uma Lista (5/11)

- **Lista é Array de Referências (1/3)**
  - Internamente, a lista é estruturada como um **array de referências**.
    - O que é armazenado é uma sequência de endereços de memória onde os elementos, de fato, residem.

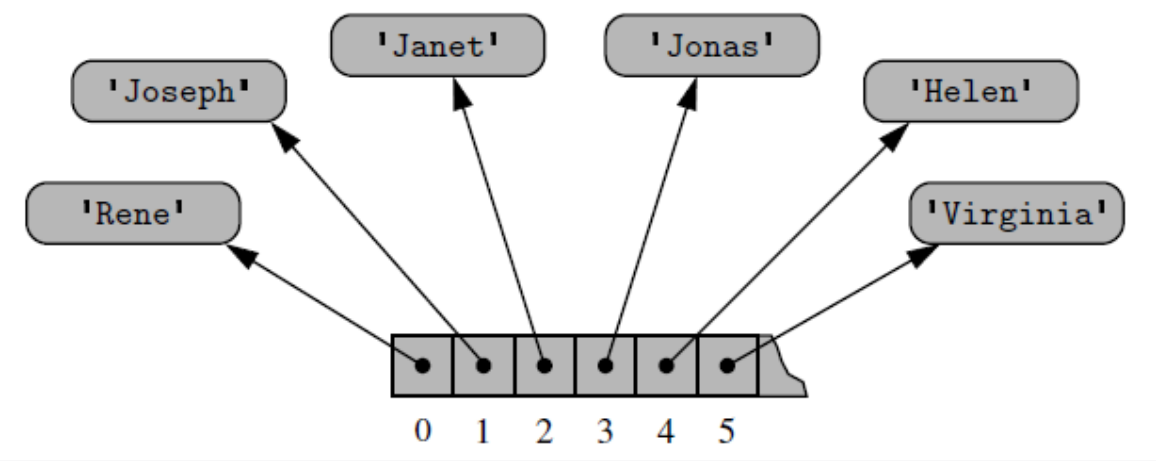
lst = ["Rene", "Joseph", "Janet", "Jonas", "Helen", "Virginia"]



Todos os exemplos deste tópico da aula foram retirados de: Goodrich et al. (2013). "Data Structures and Algorithms in Python"

Organização Interna de uma Lista (6/11)

- **Lista é Array de Referências (2/3)**
  - **Vantagem:** mesmo que o tamanho dos elementos varie, o número de bits usado em cada posição na lista é fixo (64 bits).
    - Assim, o **tempo de acesso** de um elemento pelo índice ***i*** é **constante**.
    - Para localizar um índice basta fazer: end inicial + 64 * i.
      - Onde end incial = id da lista.
  - **Desvantagem**: maior gasto de memória.
    - Preciso gastar com o array de referências e com os elementos propriamente ditos.



Organização Interna de uma Lista (7/11)

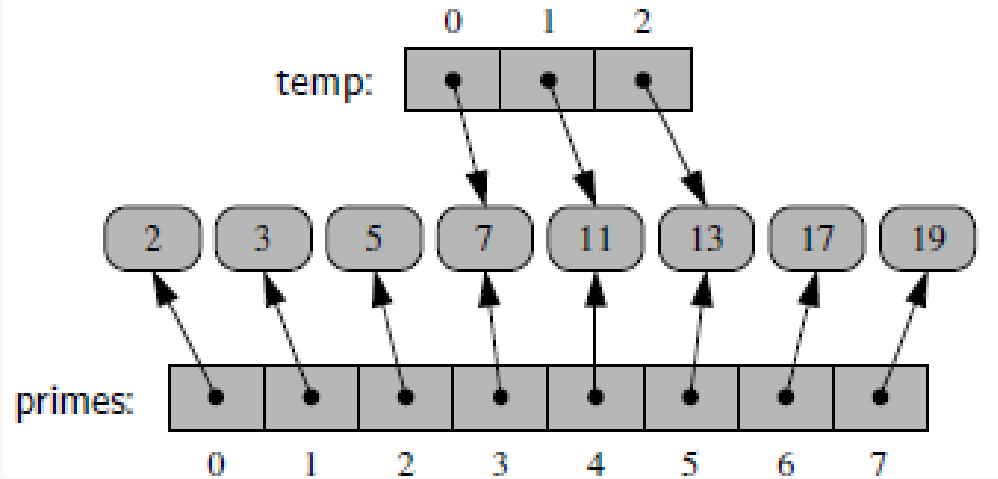
- **Lista é Array de Referências (3/3)**

```
>>> lst = ["Rene", "Joseph", "Janet", "Jonas", "Helen", "Virginia"]
>>>
>>> id(lst)
2327191884480
>>>
>>> id(lst[0])
2327191885680
>>>
>>> id(lst[1])
2327191885744
>>>
>>> id(lst[5])
2327191886064
>>>
```

Organização Interna de uma Lista (8/11)

- **Otimização de Memória (1/4)**
  - Para otimizar a ocupação de memória, o Python utiliza alguns **truques**.
  - Por exemplo, um mesmo objeto pode ser elemento de duas ou mais listas diferentes.

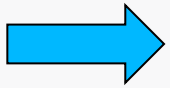
```
>>> primes = [2,3,5,7,11,13,19]
>>> temp = primes[3:6]

>>> id(primes[5])
140706767149088

>>> id(temp[2])
140706767149088

>>> id(primes)
2327191885248

>>> id(temp)
2327191885312
```

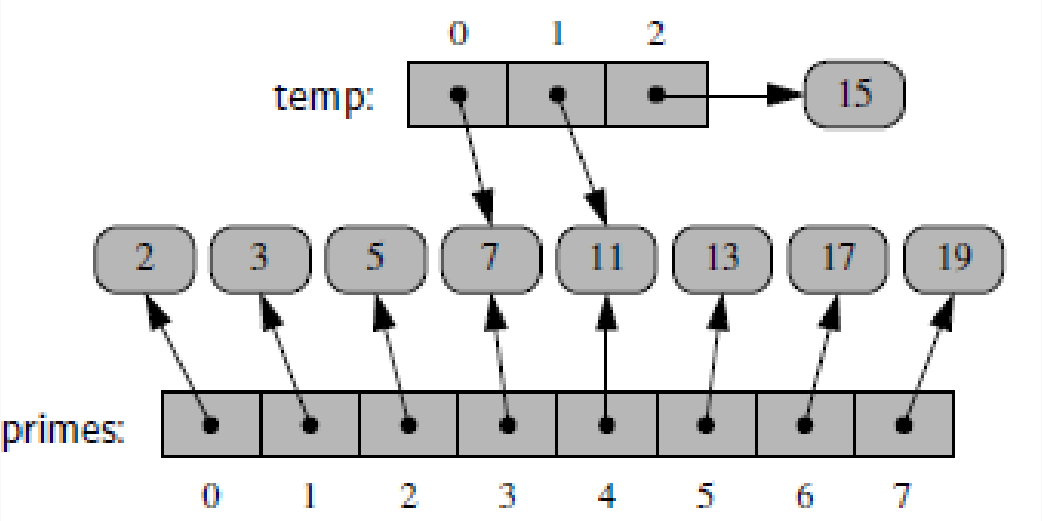


Organização Interna de uma Lista (9/11)

- **Otimização de Memória (2/4)**
  - Mas se um elemento de uma das listas é modificado, a referência muda.
  - Por exemplo, fazendo temp[2] = 15, a posição 2 de temp irá apontar para outro objeto!

```
>>> temp[2] = 15

>>> id(primes[5])
140706767149088

>>> id(temp[2])
140706767149152
```



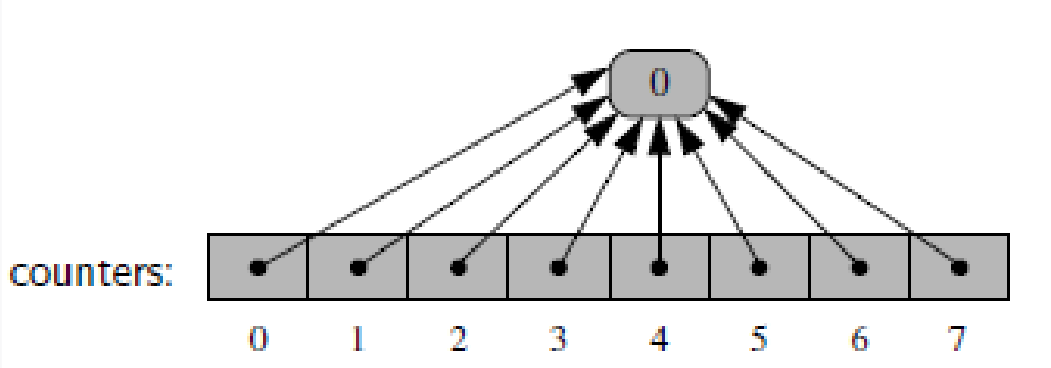


- **Em resumo**: O Python tenta otimizar ao máximo a ocupação de memória.
- Se um objeto é alterado, o Python realiza apenas os reajustes necessários em memória para resolver a operação.

Organização Interna de uma Lista (10/11)

- **Otimização de Memória (3/4)**
  - Outro exemplo interessante em Goodrich et al. (2013) é o resultado da operação de repetição.
  - Veja que no exemplo abaixo, as 8 células apontam para o mesmo objeto!

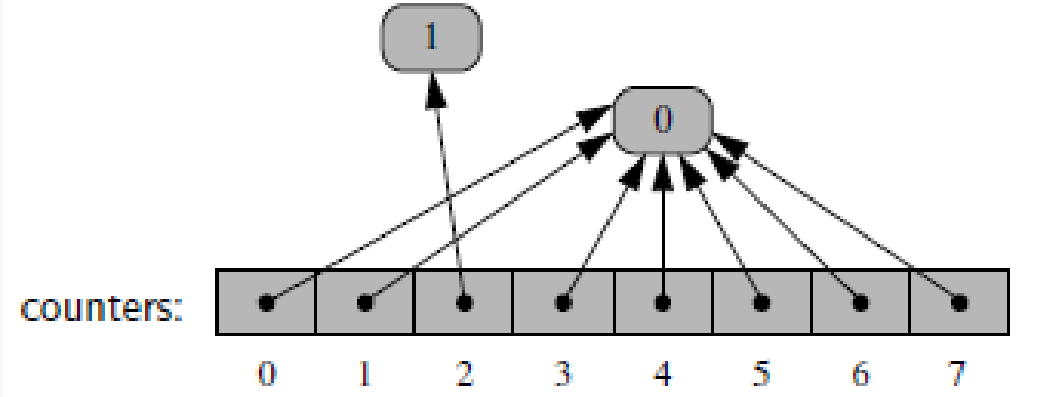
```
>>> counters = [0] * 8

>>> id(counters)
2327191885632

>>> id(counters[0])
140706767148672

>>> id(counters[7])
140706767148672

>>> id(counters[2])
140706767148672
```



Organização Interna de uma Lista (11/11)

- **Otimização de Memória (4/4)**
  - Como no exemplo anterior, se um elemento é alterado, a configuração interna em memória muda.

```
>>> counters[2] += 1

>>> id(counters)
2327191885632

>>> id(counters[0])
140706767148672

>>> id(counters[7])
140706767148672

>>> id(counters[2])
140706767148704
```



Por que o append() é O(1) (1/4)?

- **Explicação de** Goodrich et al. (2013)
  - Quando criamos uma lista no Python, ela é internamente estruturada com um determinado tamanho, maior do que o número de elementos da lista.
    - **Ex.**: o usuário cria uma lista com 4 elementos, mas o Python pode reservar 8 posições.

```
>>> w = ["John", "Yoko", "Julian", "Sean"]
```

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	
John	Yoko	Julian	Sean					*w*

    - Essa capacidade extra permite com que o *append* de um novo elemento seja feito para a próxima posição livre (há uma variável que permite acesso direto a essa posição).

** Na figura, os valores estão sendo exibidos dentro das posições da lista apenas para facilitar a visualização. Como vimos no slide anterior, na verdade a lista é um array de referências.*

Por que o append() é O(1) (2/4)?

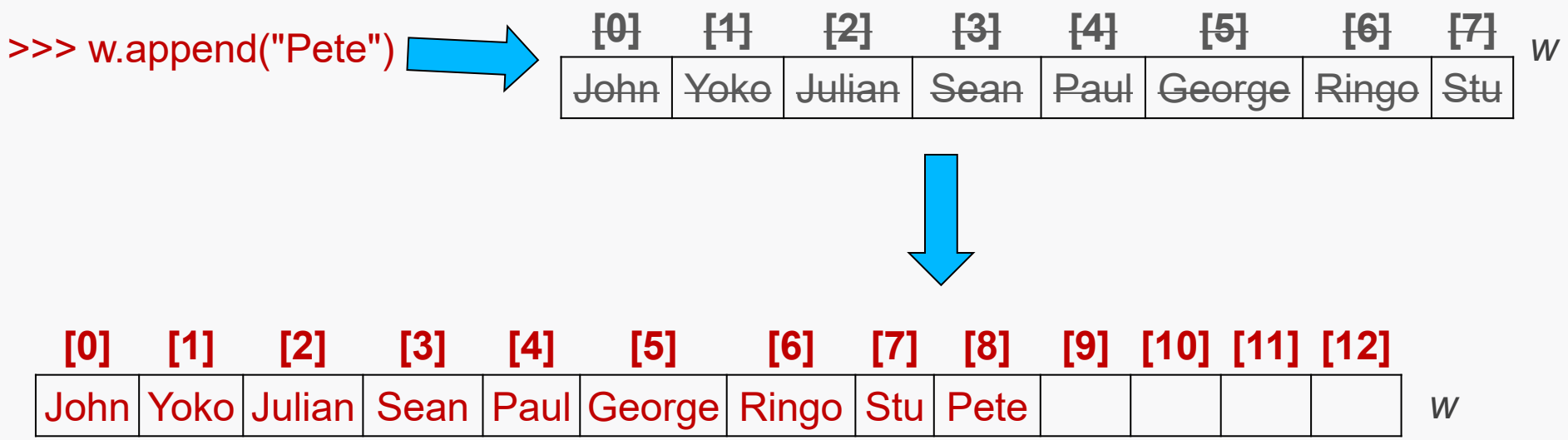
- **Explicação de** Goodrich et al. (2013)
  - Entretanto, se o usuário continuar inserindo elementos, a capacidade da lista vai estourar.
    - Veja que no exemplo abaixo, se mais um *append* for realizado, não teríamos como armazenar o elemento !!!!

```
>>> w.append("Paul")
>>> w.append("George")
>>> w.append("Ringo")
>>> w.append("Stu")
```

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	
John	Yoko	Julian	Sean	Paul	George	Ringo	Stu	*w*

Por que o append() é O(1) (3/4)?

- **Explicação de** Goodrich et al. (2013)
  - Conforme o usuário insere elementos na lista, capacidade interna eventualmente irá estourar.
  - Nesse caso, um novo espaço em memória **de tamanho maior** será alocado. Ou seja, uma nova lista será criada !!!
    - Os elementos da lista anterior serão movidos para essa nova área.
  - Apenas quando essa situação ocorre o append deixa de ter custo O(1). Nesse **pior caso**, ele se torna O(n);



Por que o append() é O(1) (4/4)?

- **Evidência empírica fornecida por** Goodrich et al. (2013)
  - No código abaixo, o tamanho da lista em bytes não aumenta a cada append.

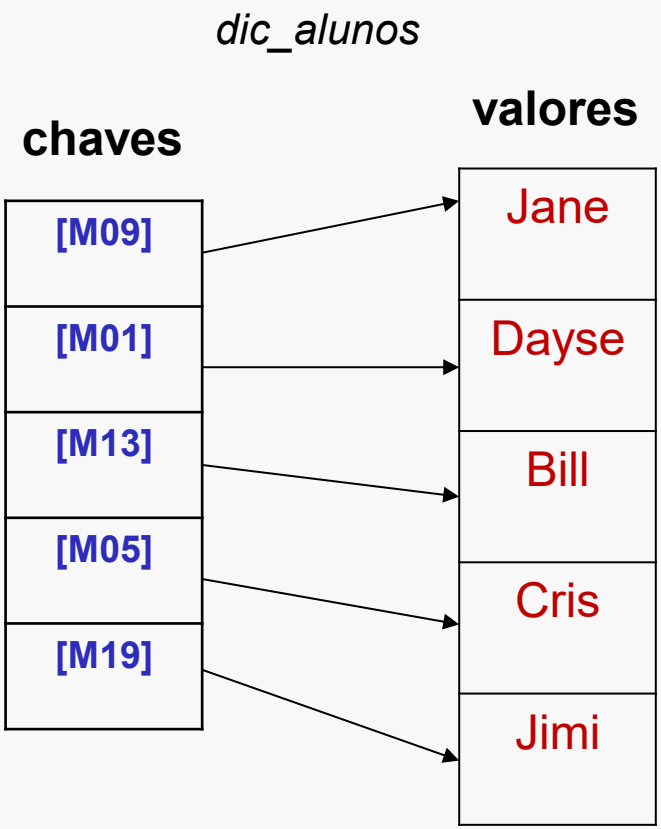
```
import sys                                  # provides getsizeof function
data = [ ]
for k in range(n):                          # NOTE: must fix choice of n
  a = len(data)                             # number of elements
  b = sys.getsizeof(data)                   # actual size in bytes
  print('Length: {0:3d}; Size in bytes: {1:4d}'.format(a, b))
  data.append(None)                         # increase length by one
```

Code Fragment 5.1: An experiment to explore the relationship between a list's length and its underlying size in Python.

```
Length:   0; Size in bytes:   72
Length:   1; Size in bytes:  104
Length:   2; Size in bytes:  104
Length:   3; Size in bytes:  104
Length:   4; Size in bytes:  104
Length:   5; Size in bytes:  136
Length:   6; Size in bytes:  136
Length:   7; Size in bytes:  136
Length:   8; Size in bytes:  136
Length:   9; Size in bytes:  200
Length:  10; Size in bytes:  200
Length:  11; Size in bytes:  200
Length:  12; Size in bytes:  200
Length:  13; Size in bytes:  200
Length:  14; Size in bytes:  200
Length:  15; Size in bytes:  200
Length:  16; Size in bytes:  200
Length:  17; Size in bytes:  272
Length:  18; Size in bytes:  272
Length:  19; Size in bytes:  272
Length:  20; Size in bytes:  272
Length:  21; Size in bytes:  272
Length:  22; Size in bytes:  272
Length:  23; Size in bytes:  272
Length:  24; Size in bytes:  272
Length:  25; Size in bytes:  272
Length:  26; Size in bytes:  352
```

Dicionário

- **Dicionário (*Dictionary*) – Definição:**
  - ED em que elementos são **pares chave:valor**.
    - A chave (*key*) identifica um item e o valor armazena o conteúdo do mesmo.
    - Qualquer valor pode ser recuperado de forma **extremamente rápida** por sua chave
      - A *chave é hasheada...*



  - Características:
    - As **chaves** devem ser **únicas**, mas podem existir **valores duplicados**.
    - **Mutável** – pode ser alterado.
    - **Iterável** – capaz de retornar seus elementos um por vez em um laço.
    - **Não é sequência** – elemento não pode ser indexado por posição.

Dicionário: Operações (1/10)

- **Operações básicas**:
  - Recuperar o valor de uma chave
  - Adicionar **entrada** (*par chave:valor*)
  - Modificar o valor de uma chave.
  - Pesquisar se chave existe
  - Remover **entrada** (*par chave:valor*)
  - Recuperar todas as chaves ou todos os valores.
  - Iterar pelas chaves, valores ou pares chave:valor

- **Obs1**: **Não temos indexação** e **nem fatiamento**, pois os elementos dos dicionários são indexados por chaves e não por posição.

- **Obs2**: O termo **entrada** (*entry*) também é utilizado para designar um **par chave:valor**.
  - Esse termo é originário dos dicionários tradicionais. https://www.macmillandictionary.com/learn/dictionary-entry.html

Dicionário: Operações (2/10)

- **Recuperando o valor de uma chave**

```
dic = {"mae":"Yoko", "pai":"John", "filho":"Sean"}

dic["mae"]          # 'Yoko'
dic['filho']        #'Sean'

>>> dic["primo"]    # Ocorre erro se especifico chave inexistente.
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'primo'
```

- **Adicionando uma nova entrada (elemento chave:valor)**

```
dic["enteado"] = "Julian"  #{'mae': 'Yoko', 'pai': 'John', 'filho': 'Sean', 'enteado': 'Julian'}
```

- **Modificando o valor de uma chave**

```
dic["pai"] = "John Lennon"   #{'mae': 'Yoko', 'pai': 'John Lennon', 'filho': 'Sean',
                              'enteado': 'Julian'}
```

Dicionário: Operações (3/10)

- **Checando se uma chave existe**

```
dic = {"mae":"Yoko", "pai":"John", "filho":"Sean"}

"mae" in dic            # True     (in = a chave pertence ao dicionário?)
"primo" in dic          # False
"primo" not in dic      # True     (not in = a chave não pertence ao dicionário?)
```

- **Tamanho do dicionário – função len()**

```
len(dic)                # 3
```

Dicionário: Operações (4/10)

- **Métodos disponíveis para Dicionários**
  - dic.**get(*k*)**: retorna o valor associado à chave *k* do dicionário. Se a chave não existir, não retorna erro.
  - dic.**pop(*k*)***:* remove o par referente à chave *k* e retorna o valor que foi removido.
  - dic.**popitem()***:* remove o último par inserido e o retorna em uma tupla (chave, valor)
  - dic.**setdefault(*k, v*)** : retorna o valor associado à chave *k*. Caso essa chave não exista no dicionário, o método vai inserir a chave *k* associada ao valor *v*.
  - dic.**clear()**: esvazia o dicionário
  - dic.**copy()**: faz uma shallow copy do dicionário. Para uma deep copy, é preciso usar o módulo copy.
  - dic.**update(**iter**)**: insere todos os elementos do iterável *iter* em *dic.* Caso alguma chave já exista, o valor será sobrescrito.
  - dic.**fromkeys(*lst_keys, v*)** : cria um dicionário contendo todas as chaves especificadas em *lst_keys*. Se *v* for especificados, todas estarão associadas ao valor *v.* Caso contrário, todas estarão associadas ao valor None.
  - dic.**items()**: retorna uma visão contendo os pares chave-valor do dicionário como uma lista de tuplas no formato [(k1, v1), (k2, v2), ..., (kn, vn)]
  - dic.**keys()**: retorna uma visão contendo as chaves do dicionário como uma lista.
  - dic.**values()**: retorna uma visão contendo os valores do dicionário como uma lista

Dicionário: Operações (5/10)

- **Inserindo e removendo entradas**

```
dic1 = {"mae":"Yoko", "pai":"John", "filho":"Sean"}
dic2 = {"bateria": "Ringo", "baixo": "Paul", "guitarra1": "George"}

dic1.pop("mae")          # remove “mae”: {'pai': 'John', 'filho': 'Sean'}
del dic1["filho"]        # com del posso também apagar 1 entrada : {'pai': 'John'}

dic1.update(dic2)        # insere os elementos de dic2 em dic1:
                         # {'pai': 'John', 'bateria': 'Ringo', 'baixo': 'Paul', 'guitarra1': 'George'}

dic1["guitarra2"] = "John"  # insere nova entrada:  {'pai': 'John', 'bateria': 'Ringo',
                            # 'baixo': 'Paul', 'guitarra1': 'George', 'guitarra2': 'John'}

dic1.pop("pai")          # remove a entrada “pai": { 'bateria': 'Ringo’, ‘baixo': 'Paul’,
                         #  'guitarra1': 'George', 'guitarra2': 'John’}

dic1.clear()             # esvazia o dicionário
```

Dicionário: Operações (6/10)

- **Recuperando o valor de uma chave (revisitado...)**

```
dic = {"mae":"Yoko", "pai":"John", "filho":"Sean"}


>>> dic["primo"]      # Usando colchetes, ocorre erro se especifico chave inexistente.

Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'primo'


>>> dic.get("primo")     # Porém com o método get(), não ocorre erro.

>>>                      # Ele apenas retorna None...
```

Dicionário: Operações (7/10)

- **Checando se uma chave existe (já havíamos mostrado)**

```
dic = {"mae":"Yoko", "pai":"John", "filho":"Sean"}

"mae" in dic          # True      (in = a chave pertence ao dicionário?)
"sogra" in dic        # False
```

- **Checando se um valor existe (não havíamos mostrado...)**

```
>>> "Yoko" in dic.values()
True

>>> "Yoki" in dic.values()
False
```

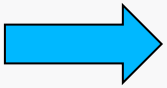
Dicionário: Operações (8/10)

- **Iterando apenas sobre as chaves**
  - Para iterar sobre as chaves, há duas formas possíveis:

1. Não usar método nenhum
2. Usar o método **keys**(), que retorna uma visão contendo a lista de chaves.

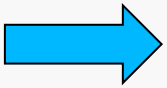
```
dic = {"mae":"Yoko", "pai":"John", "filho":"Sean"}
```

```
for chave in dic:
    print(chave)
```



```
mae
pai
filho
```

```
for chave in dic.keys():
    print(chave)
```

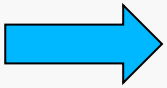


```
mae
pai
filho
```

Dicionário: Operações (9/10)

- **Iterando apenas sobre os valores**
  - Para iterar sobre os valores, usa-se o método **values()**, que retorna uma visão contendo a lista de valores.

```
dic = {"mae":"Yoko", "pai":"John", "filho":"Sean"}

for valor in dic.values():
    print(valor)
```

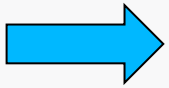


```
Yoko
John
Sean
```

Dicionário: Operações (10/10)

- **Iterando sobre chaves e valores ao mesmo tempo**
  - Mais interessante, as chaves e seus respectivos valores podem ser recuperados ao mesmo tempo usando o método **items()**
  - Com a técnica de **desempacotamento**, colocamos a chave e o valor em duas variáveis distintas.

```
dic = {"mae":"Yoko", "pai":"John", "filho":"Sean"}

for chave, valor in dic.items():
    print(chave, valor)
```



```
mae Yoko
pai John
filho Sean
```

  - **Obs.**: Até o Python 3.5, resultados poderiam vir em qualquer ordem. A partir do Python 3.6, eles vêm na ordem em que foram inseridos.
    - Entretanto, conceitualmente não existe ordem em um dicionário.

Dict Comprehension (1/2)

- **Dict Comprehension**
  - Similar ao list comprehension, porém para criar dicionários.
    - A definição deverá estar entre chaves.
    - É preciso especificar quem será a chave e quem será o valor.

Exemplo 1 – básico:

```
>>> { x: x**2 for x in (2, 4, 6) }
{2: 4, 4: 16, 6: 36}
```

Exemplo 2 – operação matemática:

```
>>> d = {'leite': 4.10, 'café': 8.99, 'pão': 3.50}
>>> reajuste = 1.1
>>> d = {produto: round(preco * reajuste,2) for (produto, preco) in d.items()}

>>> d
{'leite': 4.51, 'café': 9.89, 'pão': 3.85}
```

Dict Comprehension (2/2)

- **Dict Comprehension**

Exemplo 3 –filtragem:

```
>>> original_dict = {'jack': 38, 'michael': 48, 'guido': 57, 'john': 33}
>>> {k: v for (k, v) in original_dict.items() if v > 40}
{'michael': 48, 'guido': 57}
```

Exemplo 4 – transformação:

```
>>> original_dict = {'jack': 38, 'michael': 48, 'guido': 57, 'john': 33}

>>> {k: ('senior' if v > 40 else 'junior') for (k, v) in original_dict.items()}
{'jack': 'junior', 'michael': 'senior', 'guido': 'senior', 'john': 'junior'}
```

** Exemplos retirados de https://www.programiz.com/python-programming/dictionary-comprehension*

Função zip()

- **zip()**
  - Utilizando a função **zip()**, podemos criar um dicionário a partir de duas listas:
    - Na primeira passamos as chaves.
    - Na segunda os valores

```
>>> d = dict(zip([1, 2, 3], ["A", "B", "C"]))
>>>
>>> d
{1: 'A', 2: 'B', 3: 'C'}
```

Dicionário: Complexidade das Operações

- **Complexidade**
  - Na tabela abaixo, veja que a maioria das operações é O(1).
  - Porém, veja que indica-se O(1) para a busca (get) pela chave *k.*
    - A busca por valor (não mostrada abaixo) é O(n).

```
Dictionaries: dict
                                Complexity
Operation      | Example       | Class          | Notes
---------------+---------------+----------------+-------------------------------
Index          | d[k]          | O(1)           |
Store          | d[k] = v      | O(1)           |
Length         | len(d)        | O(1)           |
Delete         | del d[k]      | O(1)           |
get/setdefault| d.get(k)      | O(1)           |
Pop            | d.pop(k)      | O(1)           |
Pop item       | d.popitem()   | O(1)           | popped item "randomly" selected
Clear          | d.clear()     | O(1)           | similar to s = {} or = dict()
View           | d.keys()      | O(1)           | same for d.values()

Construction   | dict(...)     | O(len(...))    | depends # (key,value) 2-tuples

Iteration      | for k in d:   | O(N)           | all forms: keys, values, items
                                                | Worst: no return/break in loop
So, most dict operations are O(1).
```

Fonte: https://www.ics.uci.edu/~pattis/ICS-33/lectures/complexitypython.txt

Hash (1/11)

- **Motivação (1/4)**
  - Vamos relembrar alguns conceitos sobre **listas** apresentados no início da aula.
    - O número de bits usado em cada posição da lista é fixo (64 bits).
    - Por isso, o **tempo de acesso** de um elemento **pelo índice *i*** é **constante**.
      - Para localizar um índice basta fazer: *end inicial + 64 * i*.
        - Onde end incial = id da lista.

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[...]	[n-2]	[n-1]	
100	-1	50	535	4	-65	-7	111	...	-13	30	*u*

Hash (2/11)

- **Motivação (2/4)**
  - Considere agora outra operação:
    - **Verificar** se um dado **valor** está armazenado em uma lista não-ordenada de tamanho *n*:

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[...]	[n-2]	[n-1]	
100	-1	50	535	4	-65	-7	111	...	-13	30	*u*

```
def pertence(valor, u):
    for item in u:
        if item == valor: return True   # se encontrou, retorna True

    return False                        # senão retorna False
```

Hash (3/11)

- **Motivação (3/4)**
  - Considerando a lista *u* abaixo:
    - Para encontrar o **valor 100**, a função fará apenas **1 operação** comparação.
    - Por outro lado, para encontrar o **valor 30**, terá que fazer ***n* operações** de comparação.

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[...]	[n-2]	[n-1]	
100	-1	50	535	4	-65	-7	111	...	-13	30	*u*

```
def pertence(valor, u):
    for item in u:
        if item == valor: return True   # se encontrou, retorna True

    return False                        # senão retorna False
```

Hash (4/11)

- **Motivação (4/4)**
  - Em média, são realizadas ***n / 2* comparações** para encontrar um valor em uma **lista** qualquer. Por isso, a busca por valor na lista é O(n).
    - Logo, quanto maior *n*, maior o tempo médio.

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[...]	[n-2]	[n-1]
100	-1	50	535	4	-65	-7	111	...	-13	30

  - Entretanto, verificar se uma chave pertence a um **dicionário**, é possível realizar (quase sempre) somente **1 comparação.**
    - Ou seja: qualquer elemento é acessado pela chave em **tempo constante – O(1)** !!!
    - Isso é possível graças a técnica conhecida como **hash**.

Hash (5/11)

- **Função Hash – Definição**
  - Hash é um conceito essencial da computação, utilizado na implementação de um enorme número de algoritmos, inclusive de ciência de dados.
  - É provável que você já tenha falar sobre ele... Mas o que realmente significa? Abaixo uma definição (Leskovec et al., 2014):

Uma **função hash** *h* recebe como entrada uma ***chave*** e produz como **saída** um **número inteiro** chamado de **valor hash** (ou código de hash)

h(chave) = valor hash

Hash (6/11)

- **Função Hash – Definição**

h(chave) = valor hash

- O valor hash pode ser utilizado para definir o índice ou endereço de um elemento em uma ED.

- **Melhor explicando**:
  - Eu passo a chave para a função hash.
  - A função hash me retorna valor hash, que é um índice.
  - Então, eu uso o índice para ir direto no endereço do elemento, sem precisar fazer um laço...

h(“pai") = 0

h(“mãe") = 3

Hash (7/11)

- **A função hash() do Python (1/2)**
  - Python possui uma função **hash()** que pode ser utilizada sobre um objeto e retornará um valor inteiro.

```
>>> hash(100)
100

>>> hash(45.0)
45

>>> hash(True)
1

>>> hash(False)
0
```

```
>>> hash(3.14)
322818021289917443

>>> hash("ENCE")
2339518729674871582

>>> hash("abc")
-1124026641725669484

>>> hash( (1,2,"A","B") )
6966087159437871136
```

- O valor retornado é normalmente baseado na representação em bits do objeto.
  - *Os detalhes estão fora do escopo dessa aula. Algumas abordagens simples são apresentadas em (Leskovec et al., 2014) e outras bem mais sofisticadas em (Goodrich et al., 2013)*

Hash (8/11)

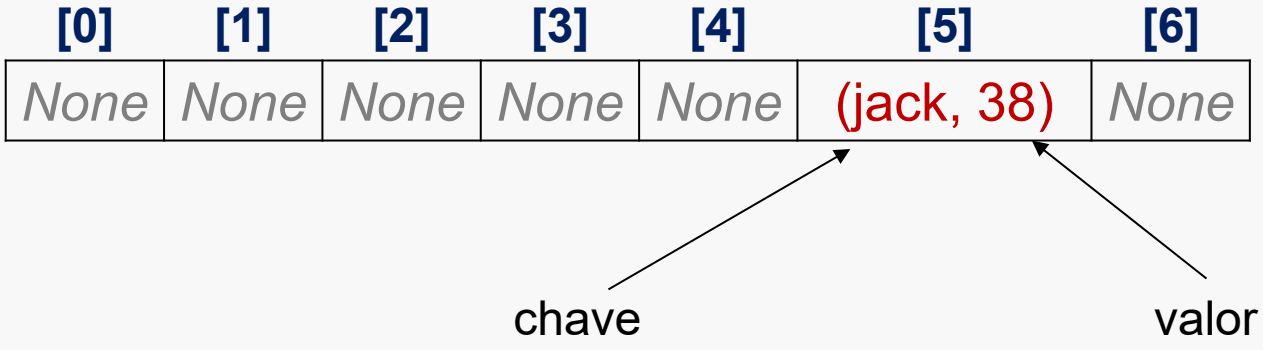
- **A função hash() do Python (2/2)**
  - **REGRA FUNDAMENTAL***: os valores hash dos objetos devem permanecer constantes durante a execução de um programa*.
    - Por esse motivo, apenas objetos imutáveis são “hasheáveis”.
    - Um objeto mutável, como uma lista, não pode ser “hasheável”, pois ao ser modificado o seu valor hash também mudaria, violando a regra fundamental.

```
>>> hash( [1,2,3,4,5] )
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unhashable type: 'list'100
```

** Chaves do tipo int, float e bool sempre produzem o mesmo valor hash independentemente do programa. No entanto, por questões relacionadas à segurança, as chaves string produzem valores diferentes a cada execução do programa (mas que permanecem constantes durante a execução do mesmo). Detalhes em (Goodrich et al., 2013)*

Hash (9/11)

- **Mas como o hash é implementado nos dicionários?**
  - Uma ED do tipo conjunto é implementada como um **HashMap** no Python e em outras linguagens.
    - O HashMap possui uma estrutura básica chamada **bucket array** para armazenar os elementos do conjunto.
      - Ou seja, “por baixo dos panos”, o bucket array é que armazena os dados do conjunto.
    - Abaixo um bucket array com 7 posições (B=7), onde apenas uma está ocupada



Hash (10/11)

- **Implementação – hash!**
  - Como você deve ter imaginado, **apenas a chave** é utilizada para **definir o índice** do elemento no bucket.
  - **Exemplo** – inserindo o par "michael":48

```
>>> val_hash = hash("michael")
>>> val_hash
-4091524431497703085

>>> val_hash % 7
3
```

[0]	[1]	[2]	[3]	[4]	[5]	[6]
None	*None*	*None*	(michael, 48)	*None*	(jack, 38)	*None*

- **Obs1**: É por isso que a chave de dicionário deve ser sempre um tipo imutável (*pois apenas os imutáveis são hasheáveis*)
- **Obs2**: Na prática é necessário tratar colisões. A colisão ocorre quando dois ou mais elementos são mapeados para o mesmo índice do bucket array. Há diferentes algoritmos para este fim.

Hash (11/11)

- **Nomenclatura**
  - Na literatura, dicionários são também são chamados por outros nomes:
    - **Arrays Associativos**
      - Visão conceitual. Expressa que um dicionário é como um array, porém com índices rotulados. O índice é a chave e cada chave está associada a um valor.
    - **Mapas**
      - Visão matemática. Indica que um dicionário pode ser visto como uma função $f(X) \rightarrow Y$ que mapeia um domínio a uma faixa de valores.
    - **Tabelas Hash**
      - Visão computacional. Indica que um dicionário é implementado com o uso da técnica hash.
    - **HashMap**
      - Nome que mistura as visões computacional e matemática.

Resumo - Listas

- **Pontos fortes**:
  - Simples e flexível.
  - Suporta muitas operações.
  - A ED mais popular do Python. *Você não consegue fazer nada em Python se não entender as listas!*
- **Pontos fracos**:
  - Pouco prática e ineficiente para operações matemáticas / estatísticas.
    - Nestes caso, melhor usar a NumPy
- **Quando usar?**:
  - Quando a maioria das buscas é por índice e as inserções podem ser feitas no final (append)
  - Quando você trabalhar com uma coleção ordenada de itens (a posição do elemento tem relevância)
  - Quando houver elementos que podem se repetir.

Resumo – Dicionário

- **Pontos fortes**:
  - Muitas aplicações práticas importantes.
  - Busca por chave tem tempo constante, independente do número de entradas do dicionário. *Implementação interna usa hash e maioria das operações é O(1).*
- **Pontos fracos**:
  - Menos operações do que as listas (ex.: não tem fatiamento)
  - Elementos não têm ordem (posição).
  - Pouco prático e ineficiente para operações matemáticas / estatísticas.
- **Quando usar?**:
  - Sempre que você tiver um conjunto não ordenado de **chaves exclusivas** que mapeie para valores.
  - Quando a maioria das consultas de seu programa for por chave.
  - Em problemas em que não faz sentido indexar o elemento por posição, mas sim por sua chave.

Referências

- Goodrich, M. T., Tamassia, R. e Goldwasser, M. H. (2013). “Data Structures and Algorithms in Python”. Wiley.

- Leskovec, J., Rajaraman, A. e Ullman, J. (2014). “Mining of Massive Datasets”. 3rd ed, Cambridge. Disponível em: http://www.mmds.org/

- Python Docs (2023). Função id(). https://docs.python.org/3/library/functions.html#id